

Java Backend Architecture

Interview Series

Chapter 10.9

Elasticsearch Performance Tuning

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

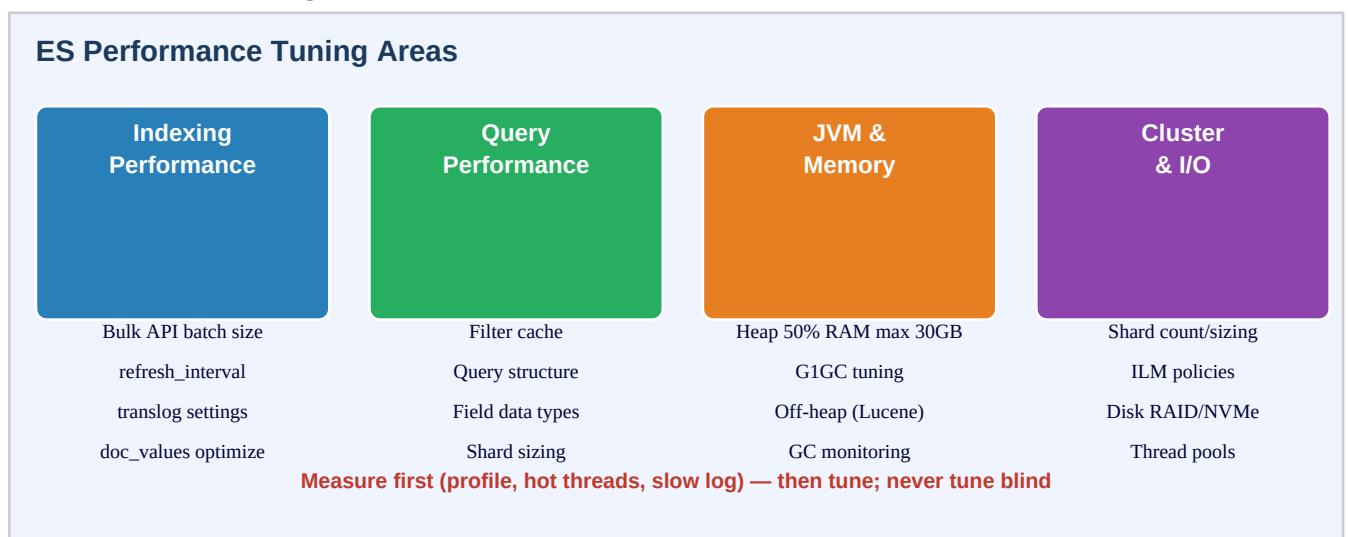
Chapter 10.9 – Elasticsearch Performance Tuning

Introduction

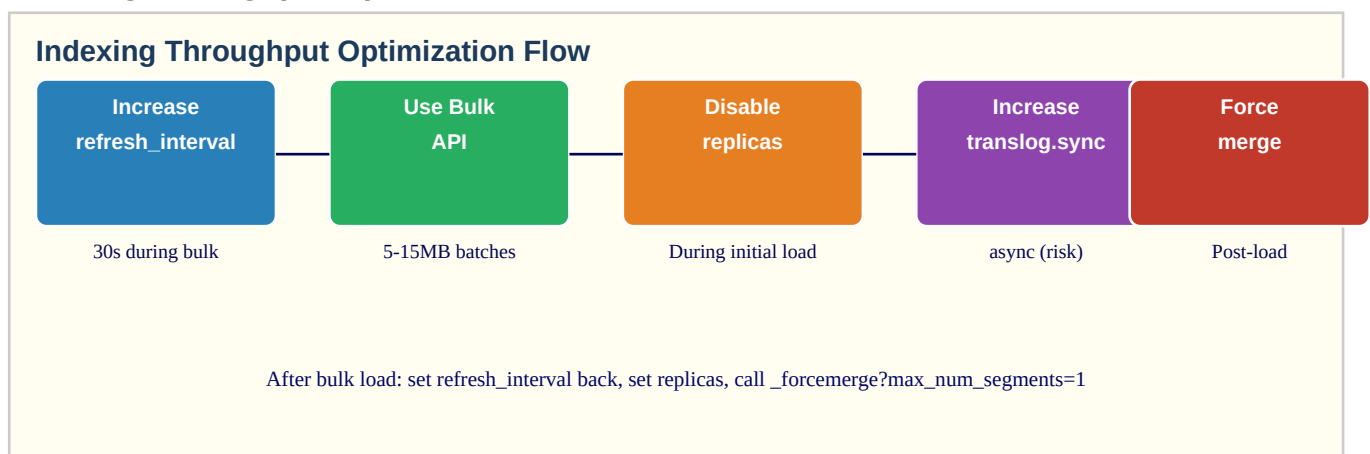
Elasticsearch performance tuning spans four areas: indexing throughput, query performance, JVM and memory management, and cluster/infrastructure configuration. Premature optimization is dangerous — always measure first using the profile API, hot_threads API, and slow log. The most common performance problems are oversharding, incorrect heap configuration, missing filter context usage, and querying without field-level filtering.

ES is built on Lucene, which uses two types of memory: JVM heap (for query caches, filter caches, fielddata) and off-heap/OS file system cache (for Lucene segment files, doc_values, stored fields). The OS file cache is just as important as heap — ES performs best when the hot data fits in the combined OS cache + heap of the nodes.

Performance Tuning Areas



Indexing Throughput Optimization



Shard Sizing Guidelines

Shard Sizing Guidelines

Primary Shards

Target 10-50 GB per shard
Default: 1 shard (ES 7+)
More shards = more parallelism
Too many = overhead (meta, memory)

Replica Shards

0 replicas during bulk load
1+ replica for HA in prod
Replicas add read throughput
Each replica = full copy of data

Rule: $\#shards \times shard_size \leq node\ RAM \times 20$ (rough heuristic for heap budget)

Oversharding is the #1 ES performance problem — start with fewer shards

Key Definitions

refresh_interval	How often ES creates a new segment from the in-memory buffer; default 1s
translog	Write-ahead log ensuring durability before segment flush; synced on each op
segment	Immutable Lucene data file; queries search across multiple segments
merge	Background process combining small segments into larger ones
force merge	Manual merge reducing shard to N segments; use only on read-only indices
bulk API	Batch multiple index/delete/update ops in a single HTTP request
filter cache	Node-level cache for frequently used filter queries; LRU, off-heap
shard request cache	Caches entire query results (size=0 agg-only queries) per shard
doc_values	Column-oriented on-disk data for sorting/aggregations; avoids fielddata
fielddata	In-heap structure for aggregations on text fields; very expensive
oversharding	Too many small shards: cluster state bloat, GC pressure, slow meta ops
hot-warm-cold arch	Tiers: hot (NVMe, all ops), warm (SSD, search only), cold (HDD, frozen)
index.codec	Compression codec: default (LZ4, fast) vs best_compression (DEFLATE, smaller)
thread_pool	ES worker threads for different operation types (search, write, get)
circuit breaker	Limits memory usage per request type to prevent OOM; throws 429 on trip

Performance Settings Reference

Setting	Default	Tuning Tip	Impact
refresh_interval	1s	Set 30s or -1 during bulk load	Indexing throughput
number_of_replicas	1	Set 0 during initial index, restore	Indexing throughput
translog.durability	request	async = ~2-3x write throughput (data loss risk)	Indexing throughput

translog.flush_threshold	512mb	Increase for write-heavy workload	Indexing throughput
index.codec	default	best_compression for archival	Index size
indices.memory.index_buffer_size	100%	Increase for write-heavy nodes	Indexing throughput
indices.queries.cache.size	10%	Increase if filter cache hit rate low	Query performance
JVM heap	auto	Set to min(50% RAM, 30GB)	Overall stability

Code Examples

1. Bulk indexing with optimal settings

```
// Step 1: Optimize index for bulk load // PUT /products/_settings { "index": {
"refresh_interval": "-1", // disable refresh during load "number_of_replicas": 0 // no replicas
during load } } // Step 2: Bulk index in batches // POST /_bulk { "index": { "_index":
"products", "_id": "1" } } { "name": "Running Shoes", "price": 89.99, "category": "footwear" }
{ "index": { "_index": "products", "_id": "2" } } { "name": "Yoga Mat", "price": 29.99,
"category": "fitness" } // Java: Bulk indexing with optimal batch size public void
bulkIndex(List<ProductDocument> products) throws IOException { int BATCH_SIZE = 500; // aim for
5-15 MB per batch for (int i = 0; i < products.size(); i += BATCH_SIZE) { List<ProductDocument>
batch = products.subList(i, Math.min(i + BATCH_SIZE, products.size())); BulkRequest.Builder
bulkBuilder = new BulkRequest.Builder(); for (ProductDocument doc : batch) {
bulkBuilder.operations(op -> op.index(idx -> idx .index("products") .id(doc.getId()))
.document(doc) ); } BulkResponse response = esClient.bulk(bulkBuilder.build()); if
(response.errors()) { response.items().stream() .filter(item -> item.error() != null)
.forEach(item -> log.error("Bulk error: {}", item.error().reason())); } } } // Step 3: Restore
settings after load // PUT /products/_settings { "index": { "refresh_interval": "1s",
"number_of_replicas": 1 } } // POST /products/_forcemerge?max_num_segments=1 // for read-only
indices
```

2. Query optimization — filter context and caching

```
// BAD: Everything in query context – no caching { "query": { "bool": { "must": [ { "match": {
"name": "running" } } }, { "term": { "category": "footwear" } } }, // should be filter { "range": {
"price": { "gte": 50 } } } }, // should be filter { "term": { "in_stock": true } } } // should be
filter ] } } // GOOD: Structured data in filter context – cached automatically { "query": {
"bool": { "must": [ { "match": { "name": "running" } } } // only full-text in query context ],
"filter": [ { "term": { "category": "footwear" } }, // cached! { "range": { "price": { "gte":
50 } } }, // cached! { "term": { "in_stock": true } } } // cached! ] } } } // Check cache
effectiveness: // GET /_nodes/stats/indices/query_cache // Look for: hit_count, miss_count,
cache_size // Hit rate > 80% is good
```

3. Slow log configuration

```
// PUT /products/_settings { "index.search.slowlog.threshold.query.warn": "10s",
"index.search.slowlog.threshold.query.info": "5s",
"index.search.slowlog.threshold.query.debug": "2s",
"index.search.slowlog.threshold.query.trace": "500ms",
"index.search.slowlog.threshold.fetch.warn": "1s",
"index.indexing.slowlog.threshold.index.warn": "10s",
"index.indexing.slowlog.threshold.index.info": "5s" } // Slow logs appear in
logs/elasticsearch_index_search_slowlog.json // Each entry includes: took, source query,
shard, node // Use this to find the top N slowest queries in production // Also useful: hot
```

```
threads API (find what's consuming CPU) // GET
/_nodes/hot_threads?threads=5&interval=500ms&type=cpu
```

4. JVM heap and GC tuning

```
# jvm.options (Elasticsearch config) # Rule: set Xms = Xmx (no heap resizing) # Rule: max 50%
of physical RAM # Rule: never exceed 30-32GB (compressed OOP pointer limit) -Xms16g -Xmx16g #
G1GC is the default GC in modern JVMs – suitable for ES # Key settings: -XX:+UseG1GC
-XX:G1HeapRegionSize=4m -XX:InitiatingHeapOccupancyPercent=30 # trigger GC earlier (default
45) -XX:G1ReservePercent=25 # Enable GC logging (important for diagnosing GC pauses)
-Xlog:gc*,gc+age=trace,safepoint:file=logs/gc.log:utctime,pid,tags:filecount=32,filesize=64m #
Monitor: GET /_nodes/stats/jvm # Look for: heap.percent, gc.collectors.young.collection_count,
gc.collectors.old.collection_count
```

5. Index lifecycle management (ILM)

```
// ILM policy: hot -> warm -> cold -> delete // PUT /_ilm/policy/products-policy { "policy": {
"phases": { "hot": { "actions": { "rollover": { "max_primary_shard_size": "50gb", "max_age":
"30d" }, "forcemerge": { "max_num_segments": 1 } } }, "warm": { "min_age": "30d", "actions": {
"shrink": { "number_of_shards": 1 }, "readonly": { } } }, "cold": { "min_age": "90d", "actions":
{ "freeze": { }, // reduces memory footprint; slower queries "searchable_snapshot": {
"snapshot_repository": "s3-repo" } } }, "delete": { "min_age": "365d", "actions": { "delete":
{ } } } } } }
```

6. Monitoring key metrics via Java

```
// Comprehensive health check in Java @Scheduled(fixedDelay = 60_000) public void
checkClusterHealth() throws IOException { // Cluster health HealthResponse health =
esClient.cluster().health(h -> h); if (health.status() != HealthStatus.Green) {
alerts.send("ES cluster health: " + health.status()); } // Index stats (search + indexing)
IndicesStatsResponse stats = esClient.indices().stats(s -> s .index("products")
.metric(List.of("search", "indexing", "merge", "refresh")) ); IndexStats indexStats =
stats.all(); log.info("Search: total={}, time={}ms", indexStats.total().search().queryTotal(),
indexStats.total().search().queryTimeInMillis()); // Node JVM NodesStatsResponse nodeStats =
esClient.nodes().stats(n -> n .metric(List.of("jvm")) );
nodeStats.nodes().values().forEach((nodeId, ns) -> { long heapPercent =
ns.jvm().mem().heapUsedPercent(); if (heapPercent > 85) { alerts.send("ES node " + nodeId + "
heap: " + heapPercent + "%"); } }); }
```

Interview Questions & Answers

Q1. What is the most common ES performance mistake?

Oversharding — having too many small shards. Each shard has overhead: it maintains its own inverted index, consumes JVM heap (for metadata), and requires coordination in scatter-gather queries. Pre-ES 7, the default was 5 primary shards, so clusters often had thousands of tiny shards. Rule: aim for 10-50 GB per shard. For most indices, 1-3 primary shards is right. Use the `_split` API to increase shard count later if data grows; use `_shrink` to reduce.

Q2. How does the refresh_interval affect performance?

ES writes documents to an in-memory buffer and periodically flushes them to a new Lucene segment — this is the 'refresh'. Default interval is 1 second (NRT). Each refresh creates a new segment, increasing segment

count over time (until background merges reduce it). Frequent refreshes = more segments = more work per query. For bulk loading: set `refresh_interval=-1` to disable, bulk load, then restore. For write-heavy production: increase to 10-30s if NRT is not required.

Q3. Explain the translog and its role in durability.

The translog (transaction log) is a write-ahead log that records every indexed document before it's flushed to a Lucene segment. On crash, ES replays the translog to recover committed writes. ``translog.durability: request`` (default) fsyncs the translog on every index operation — safe but slow. ``translog.durability: async`` fsyncs every `translog.sync_interval` (default 5s) — much faster but risks losing up to 5s of data on crash. Only use `async` for non-critical data or if data can be fully re-sourced.

Q4. What is force merge and when should you use it?

Force merge explicitly merges Lucene segments down to a specified number: ``POST /index/_forcemerge?max_num_segments=1``. Merging to 1 segment maximizes query performance by eliminating multi-segment overhead. Use ONLY on read-only (immutable) indices — time-series data after ILM rollover, archived log indices. Never force merge an actively written index: the merge process competes with indexing, causes I/O spikes, and the resulting single segment will immediately be split by new documents anyway.

Q5. What is the JVM heap recommendation for ES and why the 30GB limit?

Set heap to min(50% of RAM, 30-32GB). The 50% rule leaves the other half for Lucene's OS file cache (equally important for performance). The 30GB limit relates to JVM compressed ordinary object pointers (OOPs): below 32GB, the JVM can use 32-bit references for heap objects (saving memory and improving cache density). Above 32GB, 64-bit pointers are used, increasing memory overhead per object. Heap at 31GB is often better than 33GB.

Q6. What is G1GC and why is it the preferred GC for ES?

G1GC (Garbage First) is designed for large heaps with predictable pause times. It divides heap into equal-sized regions and collects the regions with the most garbage first. For ES: (1) it handles heap sizes 4GB-32GB well, (2) it provides shorter, more predictable GC pauses vs. CMS/ParallelGC, (3) it triggers minor GC before heap is exhausted. Key metric: old-gen GC pause time. If old-gen GCs are frequent or long (>1s), investigate fielddata, large aggregations, or heap-consuming queries.

Q7. What is the ES filter cache and how does it work?

The filter cache (node-level query cache) stores the results of frequently used filter queries as bitsets — one bit per document. Bitsets are compact and fast to AND/OR together. ES caches filter clauses that appear frequently in queries (not all filters — only after a threshold). Cache is LRU with a default size of 10% of heap. Benefits: cached filters run in microseconds on subsequent queries. Check hit rate via ``GET /_nodes/stats/indices/query_cache``.

Q8. What is fielddata and why should you avoid it?

Fielddata is an in-heap structure ES builds on demand to support aggregations and sorting on analyzed ``text`` fields. It loads all values for a field into JVM heap — for large indices this can consume gigabytes, causing GC pressure and OOM errors. Default: disabled on text fields (requires explicit ``fielddata: true`` in mapping). Solution: always use a ``keyword`` sub-field for aggregations/sorting instead of enabling fielddata on text. Only enable fielddata as a last resort for legacy systems.

Q9. How do you diagnose a slow query in ES?

1) Add ``profile: true`` to the query — returns per-shard timing breakdown of every Lucene operation (create, next_doc, advance, match, score). 2) Check the slow log (`index.search.slowlog.threshold.query.warn`). 3) Use ``GET /_nodes/hot_threads`` to see what CPU is busy with. 4) Check ``GET /index/_stats/search`` for average

query latency trends. 5) Look at circuit breaker trips: `GET /_nodes/stats/breaker``. Profile pinpoints whether the bottleneck is matching, scoring, aggregating, or fetching.

Q10. What are circuit breakers in ES?

Circuit breakers are memory safety valves that prevent operations from exceeding configured memory limits. Types: (1) ``request`` — limits memory per request (aggregations, nested). (2) ``fielddata`` — limits fielddata heap usage. (3) ``in_flight_requests`` — limits total size of all in-flight requests. (4) ``parent`` — overall limit for all child breakers. When tripped, ES returns 429 with ``circuit_breaking_exception``. Tune limits in `elasticsearch.yml` under ``indices.breaker.*``.

Q11. How does shard placement affect query performance?

ES routes queries to all relevant shards (scatter-gather). More shards = more parallel sub-queries, but also more network round trips, thread usage, and result merging overhead on the coordinating node. For small indices, one shard is faster than five. For hot data that's queried heavily, spreading across shards enables parallelism if queries scan large portions of the index. For aggregations: fewer shards means less merge overhead at the coordinator. Index per shard into dedicated data nodes for I/O isolation.

Q12. What is the OS file cache and how does it affect ES performance?

Lucene stores segment files on disk and reads them through OS buffer cache (page cache). When the working set fits in page cache, reads are served from RAM (fast). This is why the '50% heap' rule leaves half of RAM for the OS to use as page cache for Lucene files. For hot data: ensure the sum of hot segment data fits in (RAM - heap). Monitor page cache utilization with ``free -m`` — low free + low buff/cache indicates memory pressure. Use fast SSDs/NVMe to reduce the penalty of cache misses.

Q13. What is Index Lifecycle Management (ILM)?

ILM automates the transition of indices through lifecycle phases: hot (all operations), warm (read-heavy, possibly frozen), cold (infrequent access, frozen/searchable snapshot), and delete. Transitions are triggered by conditions: `max_primary_shard_size`, `max_age`, `max_docs`. Actions per phase: `rollover` (create new index), `forcemerge`, `shrink`, `readonly`, `freeze`, `searchable_snapshot`, `delete`. ILM is essential for time-series data (logs, events) to manage disk growth, query performance, and cost.

Q14. What is a frozen index and when should you use it?

A frozen index has its shards closed on disk and reloaded on demand per query. This dramatically reduces memory footprint — a frozen index uses almost no heap between queries. Query latency is higher (shard must be reopened). Use for: cold-tier data queried rarely (compliance archive, historical analytics). ES 8.x prefers ``searchable_snapshots`` over frozen indices for cold data as they store on object storage (S3) with even lower cost.

Q15. How does ``number_of_shards`` affect indexing and why can't you change it?

Shards are created at index time. Document routing uses ``hash(id) % num_primary_shards`` — changing shard count changes routing, breaking existing document locations. Primary shard count is therefore immutable after index creation. To change it: use ``_split`` API to double/triple shard count (creates a new index), or ``_shrink`` to reduce. Plan shard count upfront based on expected data volume: `shard_count = ceil(expected_size_GB / 30GB)`.

Q16. What is the bulk queue size and why does it matter?

ES has a thread pool for indexing (``write`` pool): default size = `CPU_count`, queue = 200. When all write threads are busy, requests go to the queue. When the queue is full, ES returns 429 (Too Many Requests). For high-throughput indexing: (1) increase `thread_pool.write.queue_size` in `elasticsearch.yml` (e.g., 1000). (2) Back off and retry with exponential backoff on 429 in the client. (3) Scale out by adding data nodes. Never silently drop 429 — it means data was not indexed.

Q17. How do you identify and fix mapping explosion?

Mapping explosion: too many fields in the cluster state, causing master node memory pressure. Diagnose: ``GET /index/_mapping | jq '.properties | length'`` — if thousands of fields, you have explosion. Check total cluster field count: ``GET /_cluster/stats | jq '.indices.mappings.field_types'``. Fixes: (1) Set ``dynamic: strict`` to stop new fields. (2) Use ``flattened`` type for arbitrary key-value pairs. (3) Reindex with a clean mapping. (4) Set ``indices.mapping.total_fields.limit`` (default 1000) to prevent growth.

Q18. What monitoring metrics should you watch in production?

JVM: `heap.used_percent` (alert >80%), `gc.young/old` collection time. Cluster: `status` (green/yellow/red), `unassigned_shards` (alert >0 for primary). Index: `search.query_time_in_millis/query_total` (avg latency), `indexing.index_failed`. Circuit breakers: `tripped_count` for `request/fielddata/parent`. OS: `cpu_percent`, `disk.available_in_bytes` (alert <15%). Thread pools: `write.rejected`, `search.rejected` (indicate overload). Use Kibana Stack Monitoring, Elastic APM, or Prometheus/Grafana with metricbeat.

Q19. How do you tune for high-concurrency search (many simultaneous users)?

1) Size shards to allow parallelism — more shards = more concurrent queries in parallel. 2) Increase `thread_pool.search.size` (default: $\min(\text{cpu} \times 3/2 + 1, 25)$) and `queue_size`. 3) Maximize filter cache hit rate — use filter context for structural conditions. 4) Use shard request cache for aggregation-only queries (`size=0`). 5) Add replica shards — each replica can serve reads independently. 6) Use caching layers (Redis) in front of ES for popular identical queries. 7) Load test with representative concurrent query patterns before production.

Q20. What is searchable snapshots and how does it reduce cost?

Searchable snapshots store index data in object storage (S3/GCS/Azure) and mount them as an ES index. ES reads segment files directly from the snapshot repository on demand, with local caching. Benefits: (1) eliminates the need for dedicated warm/cold nodes for old data, (2) dramatically reduces storage cost (object storage is ~10x cheaper than SSD), (3) no replica copies needed (object storage provides durability). Trade-off: higher query latency for uncached data. Ideal for cold-tier archival search.

Q21. How do you diagnose and resolve heap pressure from aggregations?

Signs: frequent old-gen GCs, `heap.used_percent` >85%, circuit breaker trips on aggs. Diagnose: profile the aggregation query, check ``_nodes/stats/breaker/request``. Fixes: (1) Use `sampler` aggregation to run aggs on a representative subset. (2) Use `execution_hint: map` instead of `global_ordinals` for terms agg on low-cardinality fields. (3) Reduce agg size (fewer buckets). (4) Pre-compute aggregations offline. (5) Upgrade nodes with more RAM. (6) Use `doc_values` instead of `fielddata` for keyword fields.